

Radar-Based Fall Detection Pipeline

A mmWave radar pipeline that detects fall events from sensor data using engineered height features and an XGBoost classifier. Designed as a proof-of-concept demo with a clear path toward real-time deployment.

Table of Contents

- [Overview](#)
 - [Project Structure](#)
 - [Model Summary](#)
 - [Quickstart](#)
 - [Pipeline Walkthrough](#)
 - [Notes & Assumptions](#)
 - [Future Improvements](#)
-

Overview

The pipeline ingests raw radar JSON data, engineers time-based vertical height (`z1`) features, and classifies whether a given sensor window contains a fall event.

Core idea: When a person falls, the radar-tracked (`z1`) height drops sharply from ~1m+ to near-floor level and stays low. The model learns this signature from engineered lag, delta, rolling, and drop features.

End-to-end flow:

Raw radar JSON → `fall_data_preprocessing.py` → Model-ready CSV → `predict_fall_from_csv.py` → FALL / NO FALL

Project Structure

File	Description
<code>feature_extraction_&_Model_Training.ipynb</code>	Exploration, feature engineering, model training, threshold selection

File	Description
<code>fall_data_preprocessing.py</code>	Cleans raw JSON and engineers features into a model-ready CSV
<code>predict_fall_from_csv.py</code>	Loads the saved model and outputs a fall/no-fall decision
<code>xg_boost_fall_detection_model.pkl</code>	Saved XGBoost classifier (threshold = 0.35)
<code>60_sec_file.ipynb</code>	Notebook to extract and visualize the demo fall snippet
<code>kitchen_fall_60sec.json</code>	60-second raw radar JSON around a clear kitchen fall
<code>kitchen_fall_60sec_preprocessed.csv</code>	Preprocessed output ready for prediction
<code>fall_prediction_results.csv</code>	Frame-level prediction output with probabilities

Model Summary

Algorithm: XGBoost Classifier

Decision threshold: 0.35 (*tuned to minimize missed falls*)

Key Features

Feature	Description
<code>z1</code>	Current vertical height of tracked target
<code>z1_lag1/2/3</code>	Height at previous 1-3 frames
<code>z1_delta1/3</code>	Frame-to-frame height change
<code>z1_drop_from_lag3</code>	Total height drop over last 3 frames
<code>z1_lead_mean5</code>	Average height over next 5 frames
<code>z1_lead_std3</code>	Height stability over next 3 frames
<code>z1_lead8</code>	Height 8 frames ahead

Feature	Description
<code>z1_is_low</code>	Binary flag: height below low-level threshold

Test Set Performance

Metric	Score
Fall Precision	81.25%
Fall Recall	86.67%
Fall F1 Score	83.87%
Macro F1	91.78%

Recall is prioritized over precision — missing a real fall is more costly than a false alert.

Quickstart

1. Install dependencies

```
bash

pip install pandas numpy scikit-learn xgboost matplotlib
```

2. Preprocess the raw JSON

```
bash

python3 fall_data_preprocessing.py \
  --data kitchen_fall_60sec.json \
  --output kitchen_fall_60sec_preprocessed.csv
```

3. Run the model

```
bash

python3 predict_fall_from_csv.py \
  --input kitchen_fall_60sec_preprocessed.csv \
  --model xg_boost_fall_detection_model.pkl \
  --threshold 0.35
```

Expected output:

Fall Prediction Summary

Input CSV: kitchen_fall_60sec_preprocessed.csv
Model file: xg_boost_fall_detection_model.pkl
Threshold used: 0.35

Final Decision

 FALL DETECTED

Results are saved to `fall_prediction_results.csv`.

Pipeline Walkthrough

Preprocessing (`fall_data_preprocessing.py`)

Transforms raw radar JSON into a clean, feature-rich CSV:

1. Expands `parsedData` into: `x1`, `y1`, `z1`, `x2`, `y2`, `z2`, `SNS`, `target_count`
2. Removes all-zero frames (empty radar readings)
3. Optionally merges fall labels from a label file
4. Maps `radarUuid` to human-readable room names
5. Creates session IDs based on timestamp gaps
6. Engineers time-based `z1` features (lag, delta, rolling stats, drop, low-height flag)

Prediction (`predict_fall_from_csv.py`)

1. Loads the preprocessed CSV
2. Loads the saved XGBoost model
3. Generates frame-level fall probabilities
4. Applies the 0.35 threshold to each frame
5. Issues a final window-level decision: **FALL DETECTED** or **NO FALL DETECTED**
6. Saves detailed per-frame results

Demo Data (`kitchen_fall_60sec.json`)

A 60-second Kitchen fall showing the expected `z1` signature:

```
20:05:00 → z1 ≈ 1.16 (standing)
20:05:05 → z1 ≈ 0.65 (beginning to fall)
20:05:09 → z1 ≈ 0.12 (impact)
20:05:13 → z1 ≈ 0.11 (on the floor)
20:05:21+ → z1 remains low
```

Notes & Assumptions

- Raw JSON must follow the same schema as the training data
 - `parsedData` must contain exactly 8 values: `x1, y1, z1, x2, y2, z2, SNS, target_count`
 - All-zero `parsedData` rows are treated as empty frames and removed
 - The current demo covers a single kitchen fall event
 - This pipeline is a proof-of-concept and is not yet production-ready
-

Future Improvements

- ☐ Train on a larger and more diverse labeled dataset (more residents, rooms, sensor placements)
- ☐ Further threshold optimization to reduce false negatives
- ☐ Add real-time streaming support
- ☐ Explore sequence models (e.g., LSTM) as more fall sequences become available
- ☐ Build a live dashboard for fall alerts and sensor monitoring
- ☐ Evaluate cross-resident and cross-room generalization